```
        if competitive(team) and competitive(opponent)
          # omitted
        else if competitive team
          # omitted
        else if competitive opponent
          # omitted
    results

  run = (teams) ->
    scored = (results for team, results of roundRobin(teams))
    ranked = rank scored
    console.log ranked
    insert ranked...
    first = ranked.slice(0, 1)[0]
    rest = ranked.slice 1
    highlight first, rest...

  { run }

sortOnPoints = (a, b) ->
  a.points > b.points

class Team
  constructor: (@name) ->
  players: 5
  compete: ->
    Math.floor Math.random()*3

window.onload = ->
  competition = makeCompetition(max:5, sort: sortOnPoints)

  disqualified = new Team "Canaries"
  disqualified.compete = null

  bizarros = ->
  bizarros.players = 5
  bizarros.compete = -> 9

  competition.run {
    wolverines : new Team "Wolverines"
    penguins: { players: 5, compete: -> Math.floor Math.random()*3 }
    injured: injured
    sparrows: new Team "Sparrows"
    bizarros: bizarros
  }
```

Code for adding the scores and goals is included in the source. It's omitted here because it's boring and not helpful for understanding the concepts.

Convert the object into an array of its values.

The Team class.

A team created with the Team class that doesn't compete. Doesn't meet the interface.

A function with the properties added. Does meet the interface.

A team created with the Team class that does meet the interface.

An object literal that meets the interface.

No `typeof` is required. Instead, the teams are tested to see whether they have the properties required for something to be considered a team *for the purposes of the competition*. Another function or module elsewhere in the program could have different expectations of what a team is. Types are not built in. Instead, consider the local requirements for objects and use them to determine whether all objects are suitable.